Week 9 — Views and indexes (plus DCL: roles, GRANT, REVOKE)

Lab: Handout lab list **#8/#9 — Views and Indexes** · lab manual Exp 8 "Indexes" and Exp 9 "Subqueries and Indexes". **DCL is not a numbered lab**, but the mini-project (handout step 7) requires it, and views are the natural place for access control, so it lives here. **Theory lectures this lines up with:** 18–20 (views); 28–33 (indexing, B+ trees), 21 for security context

Package the eight weeks of queries as named, reusable views, speed up the access paths with measured indexes, and lock the data down with roles. After this week **all five fraud personas are found by SQL views alone**: no trigger yet.

What was added

File	What it is
`sql/10_rule_views.sql`	The six rule views and `v_transaction_risk`, which adds them up and explains in plain English.
`sql/20_reporting_views.sql`	`v_active_alerts`, `v_top_risky_users`, `v_alerts_by_city_day`, `v_daily_txn_summary`.
`sql/30_indexes.sql`	Eight indexes, each justified. Two are deliberately *not* created.
`sql/40_roles_dcl.sql`	Roles `fraud_analyst`, `txn_app`, `db_admin`; three users; GRANT; default roles.
`queries/01_use_the_views.sql`	Using the views; tuning with an UPDATE; switching a rule off.
`demo/explain_before_after.sql`	`EXPLAIN ANALYZE` with and without each index (self-restoring).
`demo/dcl_demo.sh`	Seven steps from three logins: allowed, denied, REVOKE, re-GRANT, SHOW GRANTS.
`docs/benchmark_measured.md`	The real measurements, honestly interpreted.

Files in `sql/` are **loaded by the build** (`scripts/build.sh`). Files in `queries/`, `demo/` and `lab-practice/` are run by hand and are checked by `scripts/check.sh`.

What you should see

- 11 views. Rule counts: z-score **145**, off-hours **2**, new device **16**, velocity **3**, impossible travel **46** (consecutive-transaction version), merchant risk **0** (no alerts yet).
- `v_transaction_risk` explains each persona in words, e.g. 12052 → *High amount (z=6.97); New device (NEVER_SE...)*.
- `analyst_user`: allowed on `v_daily_txn_summary`, **denied** (error 1142) on `transactions`. `app_user`: can read `transactions`, denied on `users`.
- Indexes: date-range query cost **1349 → 174** (table scan → index range scan). See `docs/benchmark_measured.md` for why the account index only removes a Sort.

Concepts you must be able to explain (viva)

- **View.** A stored SELECT with no data of its own. Hides complexity, centralises logic, and is a security boundary.
- **`CREATE OR REPLACE VIEW`.** Redefines a view and keeps its grants.
- **CTE (`WITH`) and `LAG()`.** Beyond the lab list. A CTE is a named subquery for one statement; `LAG(x) OVER (PARTITION BY a ORDER BY t)` reads the previous row of the same group.

- **Index (B+ tree).** A sorted structure that lets MySQL jump to rows. Speeds reads, slows writes, costs space.
- **Composite index.** `(account_id, txn_time)` serves filters on `account_id` and on both, **not** on `txn_time` alone (leftmost-prefix rule).
- **Clustered vs secondary.** InnoDB stores the table as the primary-key B+ tree; every other index is secondary.
- **Role, GRANT, REVOKE.** A role is a named bundle of privileges. Least privilege: give each login only what its job needs.
- **DEFINER rights.** A view runs with its creator's rights, so a role can read a view without reading the tables.

More questions with full answers: `week-11-final-polish/docs/viva_questions.md`. Syntax for the closed-book lab: `docs/LAB_EXAM_CHEATSHEET.md`.

Handout step covered

Handout steps 8, 10 (views; indexing & optimisation) and 7 (DCL).

Results (captured from the running database)

Evidence: Index experiment: EXPLAIN ANALYZE

See `docs/benchmark_measured.md` for the full measured output. Key result:

Query	Without the index	With the index	Verdict
Date-range report over 6 days (`idx_txn_time`)	**Table scan**, reads all 12,067 rows, optimiser cost **1349**	**Index range scan**, reads only the 315 matching rows, cost **174**	Clear win: about 8x cheaper by cost.
Latest 50 transactions of account 1 (`idx_txn_account_time`)	index lookup on `account_id`, **then a Sort** step	index lookup that already returns rows in time order, **no Sort**	Real but tiny: an account has only ~52 rows, so sorting them costs microseconds. It would matter for accounts with millions of rows.
Merchant 50, newest 20 (`idx_txn_merchant_time`)	(not measured)	index lookup in reverse time order, stops after 20 rows	Same effect as above.